

Here is the detailed analysis of the tabular data pipeline selection process based strictly on your codebase:

1) Meta Learner

Where is it implemented?

src/tabular/meta_learner.py primarily within the MetaLearner class. **What model is it?**

It is

a **RandomForestRegressor** from sklearn.ensemble (with n_estimators=50, max_depth=6).

What are its inputs (features)?

It builds a continuous feature vector using _build_feature_vector() aggregating three contexts:

- **Task context:** Task type, domain hashing, FE budget, data quality, supervision.
- **Data profile context:** Ratios for rows, cols, missing values, class imbalance, num/cat column ratios, and boolean flags for outliers and skew.
- **Pipeline components:** Embeddings for num imputer, scaler, encoder, imbalance strategy, and boolean flags for power transform, outlier clipping, and variance filtering. **What does it predict or rank exactly?**

It predicts a floating-point score between 0.0 and 1.0 (approximating the best_score or given target metric) for a specific candidate pipeline against the current dataset characteristics. **How is it trained?**

Through MetaLearner.train_from_memory(runs). When run, it parses historical runs provided by the .json memory file, extracts the pipeline specifications and dataset profiles from all_pipelines_tested or best_pipeline, and trains on those features with the actual metric score (f1, accuracy, etc.) as the target y. **When/how is it retrained?**

There is no automated retrain hook directly in the MetaLearner, but it reads past runs and trains via train_from_memory whenever called (presumably at initialization or pipeline kick-off after fetching runs). **What does it output and how does it influence candidate pipeline selection?**

In rank_candidates(), it outputs a reordered list of the candidate pipelines. Rather than deciding candidates aggressively, it acts to soft-sort them. It calculates a final score blending the heuristic default order from pipeline generation with the meta-model's predicted score. Important note: The model is advisory only; its influence is capped (MAX_META_WEIGHT = 0.25), meaning the default heuristic ordering still strongly anchors the final ranking.

2) Similarity Logic

Where is it implemented?

src/tabular/memory_manager.py (Functions _similarity(), MemoryManager.find_good_and_bad()). **What is being compared?**

Dataset shape ratios. Specifically: missing_ratio, imbalance_ratio, num_col_ratio, and cat_col_ratio. **What similarity method is used?**

A hard-coded distance function _similarity() calculating the absolute differences across those 4 variables: $1.0 - (d_missing + d_imbalance + d_num + d_cat) / 3.0$ (with numerical count ratios weighted by 0.5). **How are similar past runs selected?**

find_good_and_bad() filters historical runs. They must match the current metric_priority and task_type family (regression or classification). The runs must possess a sim $\geq$ _SIMILARITY_THRESHOLD (0.60). Runs with best_score $\geq$ 0.60 land in the "good" list, and the rest in the "bad" list. The lists are sorted by similarity, then score. **How do these similar runs influence pipeline choices?**

In generate_pipelines():

- **Bad:** Their bad patterns (imputer/scaler/encoder/imbalance combo) are identified, and matching candidates are explicitly stripped from the generated candidate pool (_matches_bad_pattern()).
- **Good:** Up to 3 top best_pipeline specifications from "good" runs are explicitly injected into the candidate list as extra choices.

3) Static Rules + User Constraints

Where are rules implemented?

src/tabular/pipeline_generator.py within generate_pipelines(). **List key rules:**

- **Missing Vals:** median for high missing data/outliers/noisy sets; otherwise mean.
- **Scaling:** robust if the data contains outliers or is mapped to "noisy" quality; otherwise standard.
- **Imbalance:** SMOTE is allowed only if min_class_size $\geq$ 6; drops down to oversample or none depending on regression/unsupervised contexts.
- **Categoricals:** Defaults to onehot, but uses ordinal natively for high cardinality limits.
- **Transforms & Clips:** Power transform for high skew; IQR Outlier Clipper for detected outliers. **How do user constraints modify or override these rules?**

User constraints act as overrides executed by _apply_constraints(). For example, no_smote forces imbalanced samplers down to oversample. no_scaling forces none. no_power_transform/no_variance_filter/no_

outlier_clip set their respective boolean flags to False. Finally, prefer_simple applies a hard limit capping candidate variations to a maximum of 4.

4) How They Work Together

Step-by-step resolution:

1. pipeline_generator.py defines a "baseline" generic setup.
2. It generates several pipeline variants (candidate list) using the **Static Rules** built entirely using characteristics profiled from the dataset (skews, duplicates, budget limitations, outliers).
3. **Similarity Logic** filters the candidate pool. Bad runs are pruned; pipelines from similar successful runs are mapped and appended.
4. **User Constraints** are looped over and ruthlessly forced onto the candidate collection.
5. The candidates are logically deduplicated and capped in length based on FE Budget (from 4 up to 12 max).
6. Finally, the **Meta Learner** reviews the candidates and reorders them based on expected metric scores. **Which one has priority?**
7. User Constraints (Absolute highest priority)
8. Memory/Similarity (Second)
9. Static Rules
10. Meta Learner (Advisory, used solely to rank/order final candidates!) **How conflicts are avoided:**
Conflicts logically cannot occur because the resolution pipeline enforces a strict step sequence. Constraints run via explicit overrides over generated properties (d["outlier_clip"] = False) right before deduplication, ensuring they always have the final say even if Memory or Static Rules previously introduced them.

5) Final Selection

How pipelines are evaluated:

src/tabular/executor.py (evaluate_pipeline()). Each candidate acts out a Stratified K-Fold CV (up to 5 folds max based on min class count). It is evaluated across **three diverse models**: logreg (Logistic Regression), tree (Decision Tree), and gnb (Gaussian NB). This multi-model approach minimizes model-induced biases. **What metric decides the best**

pipeline:

The selected config.metric (f1, accuracy, etc.). The executor captures the mean representation across folds for each of the three models, then takes the mean of those means. When selecting the final pipeline (add_run() in memory_manager.py), it sorts by metrics[config.metric]. Ties are broken natively by maintaining original generation insertion order (simpler models are generated and evaluated first). **Where this is**

implemented:

src/tabular/executor.py returns the multi-model score aggregates, while final sorting per metric operates in memory_manager.py (add_run).

6) Memory Update**What gets stored after a run:**

All exhaustive run metadata is retained. This includes dataset details, config target, the metric_priority, holistic profile_summary metadata variables, selection_reasoning, an array of all_pipelines_tested and their derived metrics/timings, the best_pipeline configuration dictionary, and the best_score. **Where this is**

implemented:

src/tabular/memory_manager.py (under the MemoryManager.add_run() method that saves directly into memory/tabular/memory.json). **How it feeds back into meta learner +**

similarity:

The exact JSON record is appended back to memory and stored on disk. Next time train_from_memory() is fired off, the Meta Learner digests all variations inside all_pipelines_tested bridging their config to their final metric. During generation, find_good_and_bad() digests the ratios from the profile summaries to determine the next "similar dataset" to glean logic from.